

Criando classes e chamando métodos

Construtor, métodos, if/elif/else, while, for e funções — tudo no mesmo lugar.

No último encontro vimos o problema dos dados soltos, criamos a primeira classe **Personagem** (só com campos, sem construtor) e traduzimos **if/enquanto/para** do Portugol pro Python. Hoje a classe ganha um construtor de verdade e **métodos** — ações que o próprio personagem sabe fazer.

Referência rápida — antes de começar

Esse bloco não é exercício. É o modelo que os exercícios abaixo vão pedir pra reaproveitar.

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

    def mostrar_status(self):
        if self.vida >= 70:
            situacao = "saudavel"
        elif self.vida >= 30:
            situacao = "ferido"
        elif self.vida > 0:
            situacao = "critico"
        else:
            situacao = "morto"
        print(self.nome, "-", self.vida, "de vida -", situacao)

    def receber_dano(self, quantidade):
        self.vida = self.vida - quantidade
        if self.vida < 0:
            self.vida = 0

thoric = Personagem("Thoric", 100, 15)
elara = Personagem("Elara", 60, 25)

thoric.mostrar_status()
elara.mostrar_status()

thoric.receber_dano(50)
thoric.mostrar_status()
```

- `__init__(self, ...)` roda sozinho assim que `Personagem(...)` é chamado — não precisa mais preencher campo por campo como no Dia 1.
- `self` é o próprio objeto sendo criado ou usado — `self.nome` é o nome *desse* personagem específico.
- `mostrar_status()` usa `if/elif/else` pra decidir o que imprimir, dependendo da vida atual.
- `receber_dano()` muda `self.vida` e usa `if` pra impedir vida negativa.

PARTE 1

Criando classes e chamando métodos

Cada exercício usa a classe `Personagem` de referência acima como ponto de partida, ou pede uma classe nova. Reaproveitem o que já escreveram sempre que der.

Exercício 1

PERSONAGEM COMPLETO

Crie a classe `Personagem` com `__init__(self, nome, vida, ataque)` e um método `mostrar_status()` que classifica a vida em saudável / ferido / crítico / morto, como no exemplo. Crie dois personagens diferentes e chame `mostrar_status()` em cada um.

REQUISITOS MÍNIMOS

- `__init__` recebe nome, vida e ataque e guarda tudo em `self`
- `mostrar_status()` usa `if/elif/else` — pelo menos 3 faixas diferentes
- Dois objetos criados com valores diferentes

Exercício 2

MÉTODO QUE MUDA O ESTADO

Adicione o método `receber_dano(quantidade)` que subtrai `quantidade` de `self.vida`. Se o resultado ficar menor que zero, a vida fica em 0 (o personagem não pode ter vida negativa). Aplique dano duas vezes seguidas num personagem, chamando `mostrar_status()` depois de cada uma.

REQUISITOS MÍNIMOS

- `receber_dano(quantidade)` recebe um parâmetro além de `self`
- Usa `if` pra impedir vida negativa
- Chamado pelo menos duas vezes, com `mostrar_status()` confirmando o resultado

Exercício 3

CLASSE NOVA + FOR DENTRO DO MÉTODO

Crie uma classe `Inimigo` com `__init__(self, nome, tipo)`. Adicione um método `atacar_varias_vezes(vezes)` que usa um `for` pra imprimir "*nome* ataca!" o número de vezes indicado. Crie uma lista com 3 inimigos diferentes e, com outro `for` por fora da classe, chame `atacar_varias_vezes(2)` em cada um.

REQUISITOS MÍNIMOS

- Classe `Inimigo` com `__init__` próprio, diferente de `Personagem`
- `atacar_varias_vezes(vezes)` usa `for` dentro do método
- Uma lista com 3 inimigos, percorrida com `for` fora da classe

Exercício 4

MÉTODO QUE DEVOLVE UM VALOR

Ainda em `Personagem`, crie `calcular_dano_total(tipo_ataque)`, que recebe uma string (`"normal"`, `"critico"` ou `"fraco"`) e **devolve** (`return`) o valor de `self.ataque` multiplicado por 2 se for crítico, por 0.5 se for fraco, ou o valor normal nos outros casos — usando `if/elif/else`. Chame o método guardando o resultado numa variável, e só depois imprima essa variável.

REQUISITOS MÍNIMOS

- Usa `return`, não `print`, dentro do método
- `if/elif/else` cobrindo os três casos
- O resultado é guardado numa variável antes de ser impresso

PARTE 2

Classes específicas: Guerreiro e Mago

Guerreiro e Mago não têm nada em comum com Personagem por enquanto — são classes totalmente separadas, cada uma com seus próprios campos e métodos. (Herança vem depois.)

Exercício 5

GUERREIRO · WHILE

Crie uma classe `Guerreiro` com `__init__(self, nome, energia)`. Adicione um método `atacar_com_espada()` que usa um `while: enquanto self.energia > 0`, imprime uma mensagem de golpe e subtrai 1 da energia a cada volta. Quando a energia acabar, imprima que o guerreiro está cansado.

REQUISITOS MÍNIMOS

- `while` usando `self.energia` como condição
- `self.energia` diminui a cada volta do laço, dentro do método
- Mensagem final fora do `while`, quando a energia chega a 0

Exercício 6

MAGO · WHILE

Crie uma classe `Mago` com `__init__(self, nome, mana)`. Adicione um método `conjurar_ate_acabar()` que usa um `while: enquanto self.mana >= 10`, conjura um feitiço (imprime e subtrai 10 da mana). Quando não houver mais mana suficiente, imprima quantos feitiços foram conjurados no total.

REQUISITOS MÍNIMOS

- `while` com `self.mana >= 10` como condição
- Um contador (variável separada) somando a cada feitiço conjurado
- Mensagem final com o total, fora do `while`

Exercício 7 · desafio

FUNÇÃO POR FORA DAS CLASSES

Escreva uma função `duelo(guerreiro, mago)` **fora de qualquer classe**, recebendo um objeto `Guerreiro` e um objeto `Mago`. Dentro da função, compare `guerreiro.energia` com `mago.mana` usando `if/elif/else` e imprima quem "venceria" o duelo (ou empate). Chame a função passando um guerreiro e um mago que vocês criaram.

REQUISITOS MÍNIMOS

- `duelo(guerreiro, mago)` é uma função comum, não um método — não fica dentro de nenhuma classe
- Recebe dois objetos de classes diferentes como parâmetros
- Usa `if/elif/else` pra decidir o resultado